

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder

# — Load Dataset —
df = pd.read_csv("/content/credit_card_fraud_10k.csv")

# — Basic Preprocessing —
# Assume target column is "is_fraud"
X = df.drop("is_fraud", axis=1)
y = df["is_fraud"]

# Encode categorical features
for c in X.select_dtypes("object").columns:
    X[c] = LabelEncoder().fit_transform(X[c].astype(str))

# Fill missing values
X = X.fillna(X.mean())

# — Split Data —
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# — Model —
model = DecisionTreeClassifier(max_depth=5)
model.fit(X_train, y_train)

# — Prediction —
y_pred = model.predict(X_test)

# — Evaluation —
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
cm = confusion_matrix(y_test, y_pred)

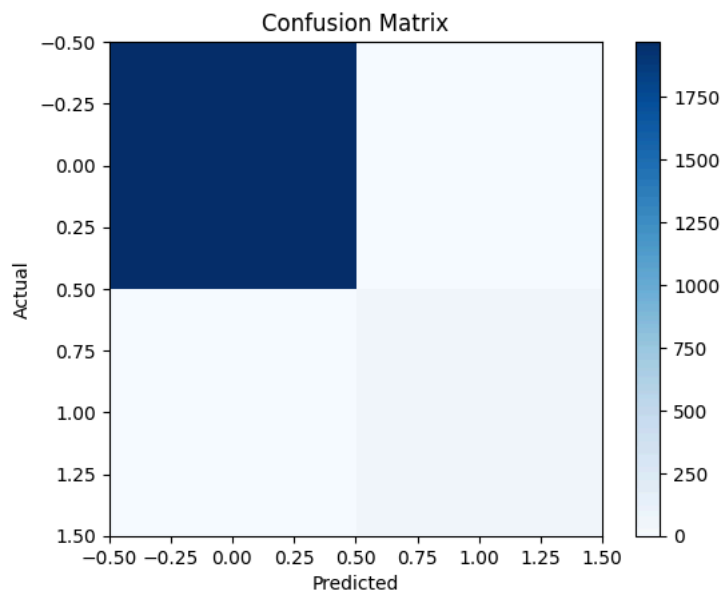
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
print("Confusion Matrix:\n", cm)

# — Plot Confusion Matrix —
plt.imshow(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.colorbar()
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```

Accuracy: 0.997
Precision: 1.0
Recall: 0.8064516129032258
F1 Score: 0.8928571428571429
Confusion Matrix:
[[1969    0]
 [   6   25]]

```



```

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder # Import LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# --- Load Dataset ---
df = pd.read_csv("/content/credit_card_fraud_10k.csv")

# --- Preprocessing ---
X = df.drop("is_fraud", axis=1)
y = df["is_fraud"]

# Encode categorical features (added this step)
for c in X.select_dtypes("object").columns:
    X[c] = LabelEncoder().fit_transform(X[c].astype(str))

# Fill missing values
X = X.fillna(X.mean())

# Scale data (important for Neural Network)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# --- Train Test Split ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# --- Simple Neural Network ---
model = MLPClassifier(hidden_layer_sizes=(50,), max_iter=200)

model.fit(X_train, y_train)

# --- Prediction ---
y_pred = model.predict(X_test)

# --- Metrics ---
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

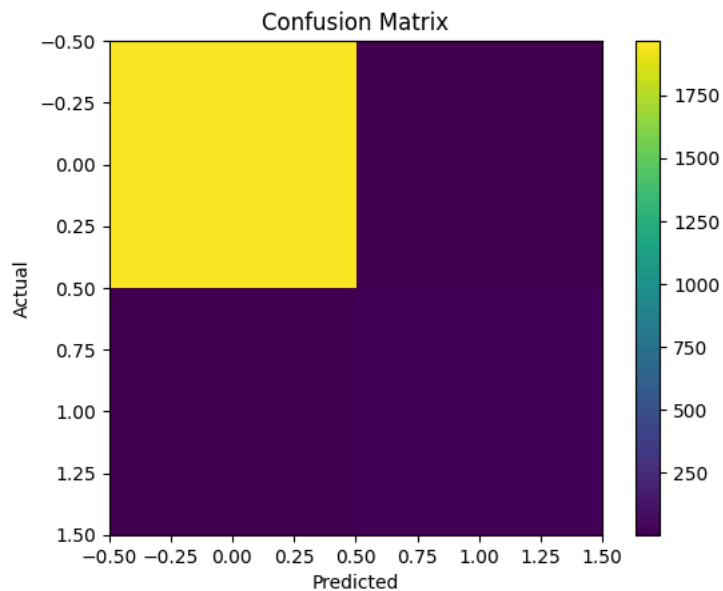
# --- Print Results ---
print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall :", recall)

```

```
print("F1 Score :", f1)
print("Confusion Matrix:\n", cm)

# — Simple Confusion Matrix Plot —
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()
```

```
Accuracy : 0.993
Precision : 0.84
Recall : 0.6774193548387096
F1 Score : 0.75
Confusion Matrix:
[[1965  4]
 [ 10  21]]
```



```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# — Load Dataset —
df = pd.read_csv("/content/credit_card_fraud_10k.csv")

# — Preprocessing —
X = df.drop("is_fraud", axis=1)
y = df["is_fraud"]

# Encode categorical features
for c in X.select_dtypes("object").columns:
    X[c] = LabelEncoder().fit_transform(X[c].astype(str))

# Fill missing values
X = X.fillna(X.mean())

# Scale data (important for KNN)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# — Train Test Split —
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# — Simple KNN Model —
model = KNeighborsClassifier(n_neighbors=5)

model.fit(X_train, y_train)

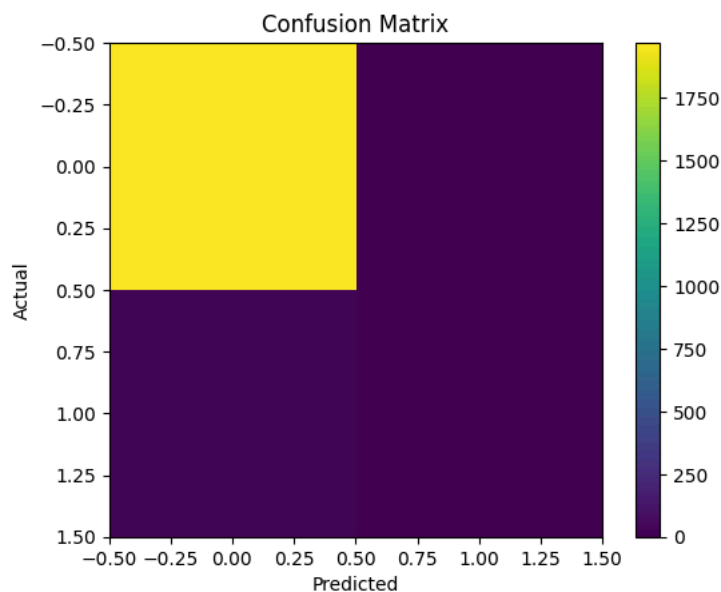
# — Prediction —
y_pred = model.predict(X_test)
```

```
# — Metrics —
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# — Print Results —
print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall :", recall)
print("F1 Score :", f1)
print("Confusion Matrix:\n", cm)

# — Simple Confusion Matrix Plot —
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()
```

```
Accuracy : 0.988
Precision : 1.0
Recall : 0.22580645161290322
F1 Score : 0.3684210526315789
Confusion Matrix:
[[1969  0]
 [ 24  7]]
```



```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder # Import LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# — Load Dataset —
df = pd.read_csv("/content/credit_card_fraud_10k.csv")

# — Preprocessing —
X = df.drop("is_fraud", axis=1)
y = df["is_fraud"]

# Encode categorical features (added this step)
for c in X.select_dtypes("object").columns:
    X[c] = LabelEncoder().fit_transform(X[c].astype(str))

# Fill missing values
X = X.fillna(X.mean())

# Scale data (IMPORTANT for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# — Train Test Split —
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42
)

# — Simple SVM Model —
model = SVC(kernel="rbf")

model.fit(X_train, y_train)

# — Prediction —
y_pred = model.predict(X_test)

# — Metrics —
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# — Print Results —
print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall :", recall)
print("F1 Score :", f1)
print("Confusion Matrix:\n", cm)

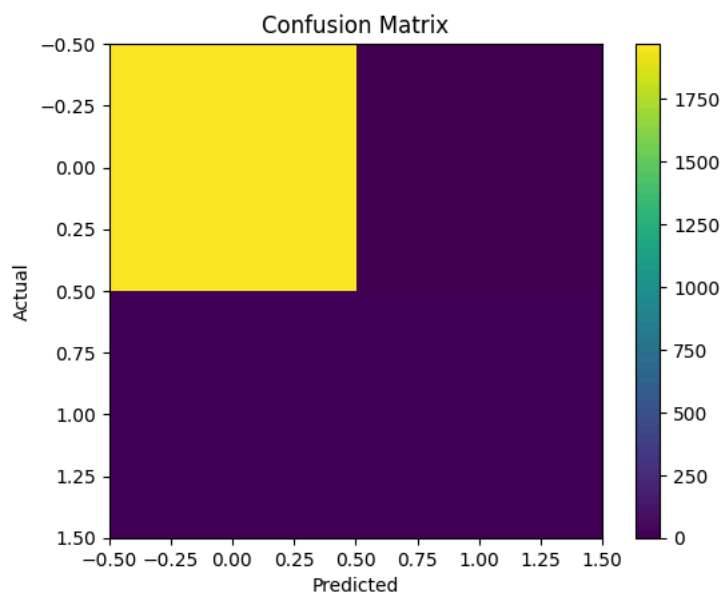
# — Simple Confusion Matrix Plot —
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()

```

```

Accuracy : 0.989
Precision : 1.0
Recall : 0.2903225806451613
F1 Score : 0.45
Confusion Matrix:
[[1969  0]
 [ 22  9]]

```



```

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder # Import LabelEncoder

# — Load Dataset —
df = pd.read_csv("/content/credit_card_fraud_10k.csv")

# — Preprocessing —
X = df.drop("is_fraud", axis=1)
y = df["is_fraud"]

# Encode categorical features (added this step)
for c in X.select_dtypes("object").columns:

```

```

X[c] = LabelEncoder().fit_transform(X[c].astype(str))

# Fill missing values
X = X.fillna(X.mean())

# — Train Test Split —
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# — Simple Random Forest —
model = RandomForestClassifier(n_estimators=100)

model.fit(X_train, y_train)

# — Prediction —
y_pred = model.predict(X_test)

# — Metrics —
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# — Print Results —
print("Accuracy :", accuracy)
print("Precision :", precision)
print("Recall :", recall)
print("F1 Score :", f1)
print("Confusion Matrix:\n", cm)

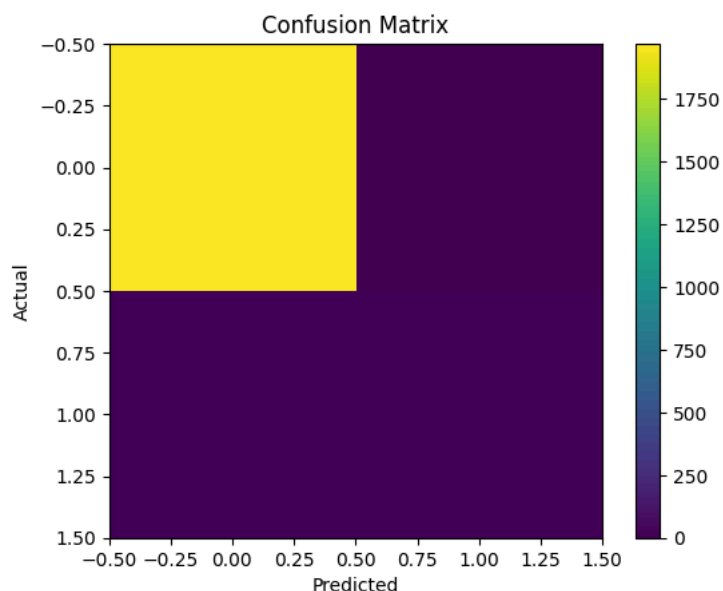
# — Simple Confusion Matrix Plot —
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()

```

```

Accuracy : 0.9955
Precision : 1.0
Recall : 0.7096774193548387
F1 Score : 0.8301886792452831
Confusion Matrix:
[[1969  0]
 [ 9  22]]

```



Start coding or generate with AI.

Random Forest performed best because it reduces overfitting and handles complex data better.

Why not KNN? → Slow for large datasets

Why not SVM? → Computationally expensive

Why Neural Network? → Good but needs more data and tuning

Why Decision Tree? → Simple but overfits

```

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
metrics_df = pd.DataFrame({
    'Model': results.keys(),
    'Accuracy': [r['Accuracy'] for r in results.values()],
    'Precision': [r['Precision'] * 100 for r in results.values()], # Convert to percentage
    'Recall': [r['Recall'] * 100 for r in results.values()], # Convert to percentage
    'F1 Score': [r['F1'] * 100 for r in results.values()] # Convert to percentage
})

metrics_df_melted = metrics_df.melt(id_vars='Model', var_name='Metric', value_name='Score')

plt.figure(figsize=(14, 7))
sns.barplot(x='Model', y='Score', hue='Metric', data=metrics_df_melted,
            palette='viridis')

plt.title('Comparative Analysis of ML Models on Credit Card Fraud Detection', fontsize=16, fontweight='bold')
plt.ylabel('Score (%)', fontsize=12)
plt.xlabel('Machine Learning Model', fontsize=12)
plt.ylim(0, 105) # Set y-limit to 0-105% for scores
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.legend(title='Metric', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```

